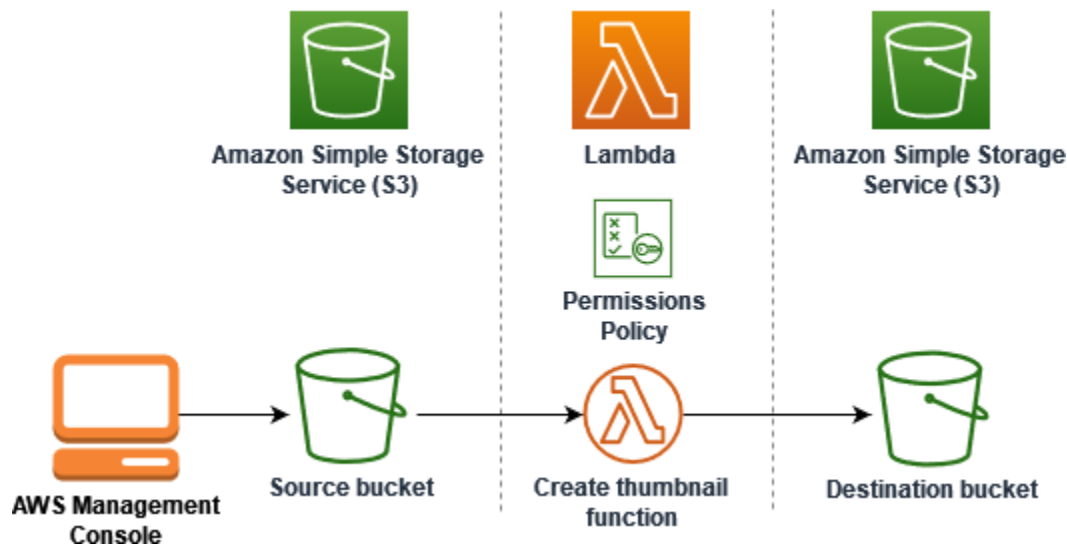


# Case Study: AWS Lambda-S3 Image Resizing using Python

## 1. Introduction

**Case Study Overview:** This case study demonstrates the implementation of an automated image resizing system using AWS Lambda and Amazon S3. By configuring a serverless architecture, the project enables dynamic resizing of images uploaded to an S3 bucket, generating thumbnails which are automatically stored in a different S3 bucket. The entire process is event-driven, triggered by the upload of an image, which invokes the Lambda function. This case study highlights the power of cloud-native technologies and serverless computing, eliminating the need for managing infrastructure while ensuring scalability and efficiency.



**Key Feature and Application:** The standout feature of this project is its seamless automation of image resizing using an event-driven approach. As soon as an image is uploaded to the designated S3 bucket, the Lambda function is invoked to process the image, resize it, and save a smaller thumbnail version in the destination bucket. This is especially useful in scenarios where optimized image delivery is critical, such as websites, mobile applications, and content delivery networks (CDNs). The resized images reduce bandwidth usage and improve page loading times, resulting in better user experiences.

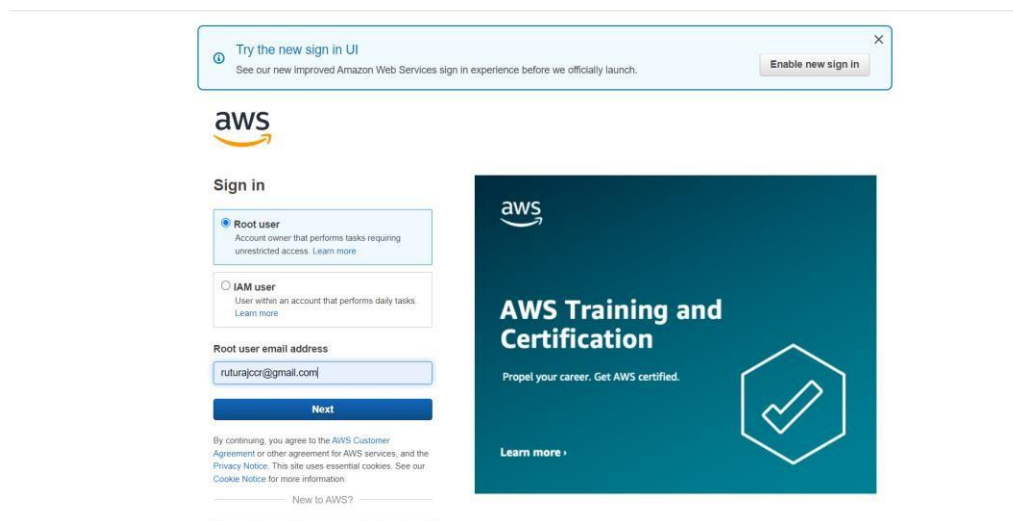
Furthermore, the solution is highly scalable, as AWS Lambda automatically handles the scaling of functions in response to increased traffic, without requiring any manual intervention. It is also cost-effective, as you only pay for the compute time you use, making it ideal for applications with fluctuating demand. By leveraging fully managed services like S3 and Lambda, this architecture not only ensures high availability but also reduces the complexity of managing backend servers and storage systems.

## 2. Step-by-Step Explanation

### Step 1: Initial Setup – Creating S3 Buckets

When you sign up for an AWS account, an *AWS account root user* is created. The root user has access to all AWS services and resources in the account. As a security best practice, assign administrative access to a user, and use only the root user to perform [tasks that require root user access](#).

Sign in to the [AWS Management Console](#) as the account owner by choosing Root user and entering your AWS account email address. On the next page, enter your password. For help signing in by using root user, see [Signing in as the root user](#) in the *AWS Sign-In User Guide*.



The screenshot displays the AWS sign-in interface. At the top, a notification banner promotes the 'new sign in UI'. Below this, the 'Sign in' section offers two options: 'Root user' (selected) and 'IAM user'. The 'Root user' option is described as the 'Account owner that performs tasks requiring unrestricted access'. A text field for the 'Root user email address' contains the email 'nriturajcc@gmail.com'. A 'Next' button is positioned below the email field. To the right of the sign-in form is a large teal banner for 'AWS Training and Certification' with a checkmark icon and a 'Learn more' link. At the bottom of the sign-in form, there is a link for 'New to AWS?'.



## Root user sign in

Email: atharvnikam123@gmail.com

Password

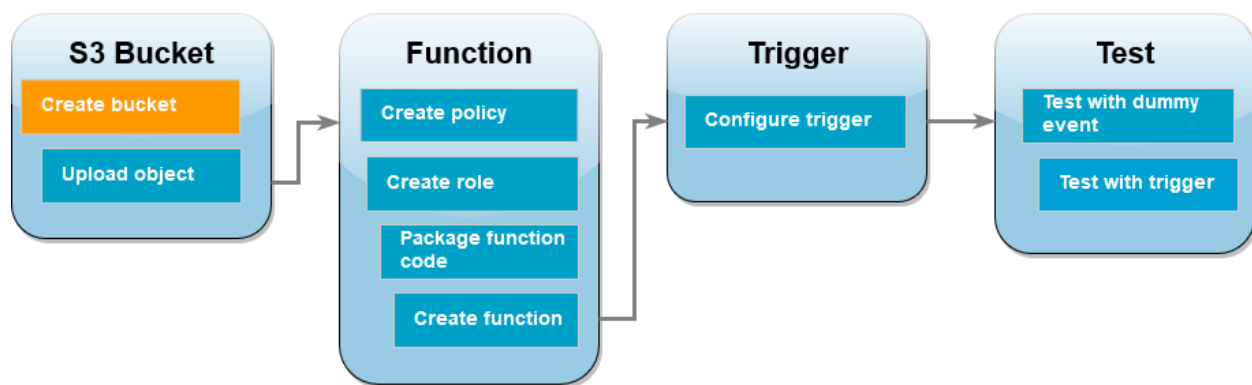
[Forgot password?](#)

.....

Sign in

[Sign in to a different account](#)

[Create a new AWS account](#)



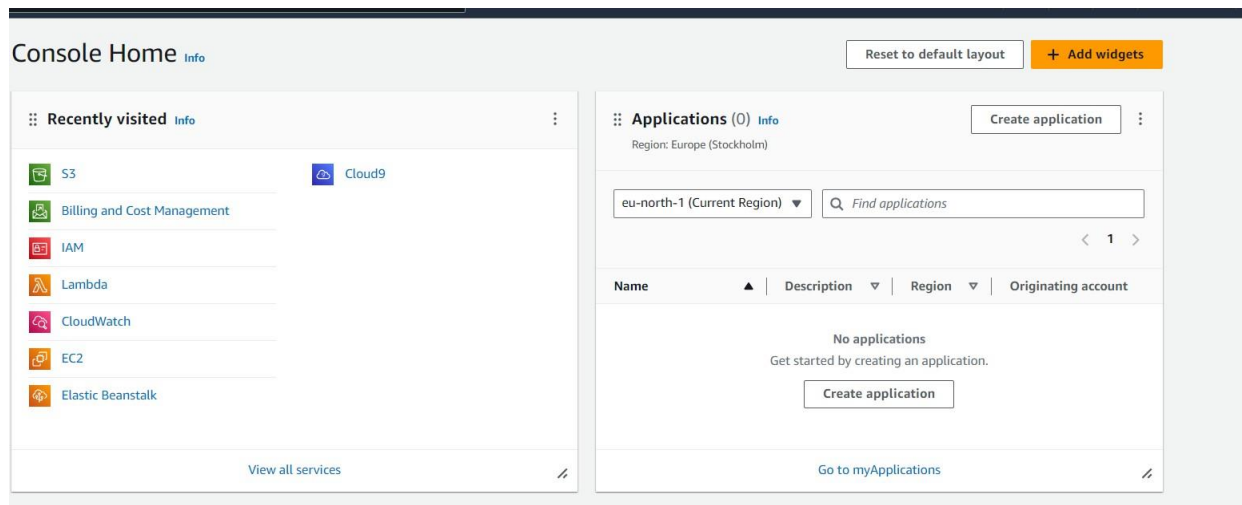
### Create Source and Destination Buckets:

I created two S3 buckets – one for uploading the original images (source bucket) and another to store the resized images (destination bucket).

*Example:*

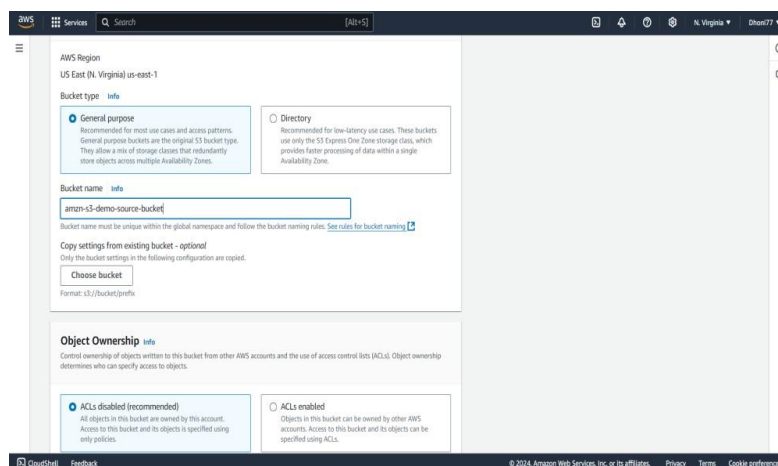
- Source Bucket: amzn-s3-bbp-source-bucket
- Destination Bucket amzn-s3-bbp-source-bucket-resized

**Open the S3 Management Console:** Start by navigating to the [Amazon S3 console](#). This is where you'll create and configure your S3 buckets.



## Create a Source Bucket:

- Choose **Create Bucket**.
- In the **General Configuration** section, provide a globally unique name for your bucket, adhering to the Amazon S3 bucket naming rules. The name should contain only lowercase letters, numbers, dots (.), and hyphens (-). For example:
- Select the AWS Region that is geographically closest to you. Ensure that your Lambda function is created in the same region later in the process.
- Leave the other settings at their default values and click **Create Bucket**.



After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket

## Create a Destination Bucket:

- Repeat the above steps to create a destination bucket, which will store the resized images..

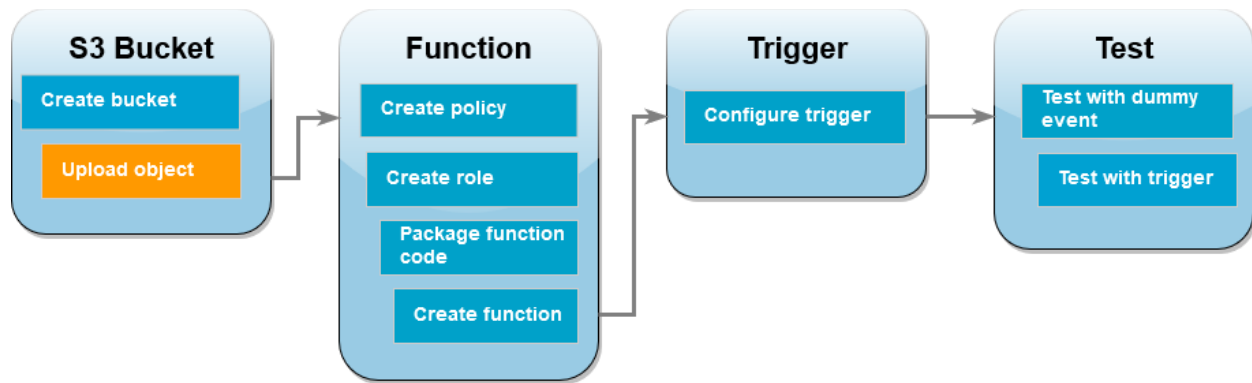
The screenshot shows the AWS Management Console 'Create bucket' page. The 'AWS Region' is set to 'US East (N. Virginia) us-east-1'. Under 'Bucket type', 'General purpose' is selected, with a note that it's recommended for most use cases and allows a mix of storage classes. The 'Directory' option is unselected. The 'Bucket name' field contains 'amzn-s3-demo-source-bucket'. Below this, there's a link to 'See rules for bucket naming'. The 'Copy settings from existing bucket - optional' section has a 'Choose bucket' button. The 'Object Ownership' section has 'ACLs disabled (recommended)' selected, with a note that all objects are owned by the account. The 'ACLs enabled' option is unselected. The footer shows '© 2024, Amazon Web Services, Inc. or its affiliates.' and links for 'Privacy', 'Terms', and 'Cookie preferences'.

### ► Advanced settings

After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket



## Uploading a Test Image to the Source Bucket (via AWS Console)

Once the Lambda function is set up, you'll need to test its functionality by uploading an image file. This is an essential step to ensure the Lambda function is triggered and processes the image as expected.

### Using the AWS Management Console:

1. Open the **Amazon S3** console and navigate to the **Buckets** page.
2. Select the **source bucket** you previously created.
3. Click on **Upload**.
4. Choose **Add files** and select any **JPG or PNG** file you want to test.
5. After selecting the file, click **Open**, then choose **Upload**.

Amazon S3 > Buckets > atharv-s3-source-bucket > Upload

## Upload [Info](#)

Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDK or Amazon S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose **Add files** or **Add folder**.

**Files and folders (1 Total, 56.1 KB)** Remove Add files Add folder

All files and folders in this table will be uploaded.

< 1 >

| <input type="checkbox"/> | Name         | Folder |
|--------------------------|--------------|--------|
| <input type="checkbox"/> | football.jpg | -      |

Upload succeeded  
View details below.

Upload: status

Close

The information below will no longer be available after you navigate away from this page.

Summary

Destination  
s3://atharv-s3-source-bucket

Succeeded  
1 file, 56.1 KB (100.00%)

Failed  
0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (1 Total, 56.1 KB)

Find by name

< 1 >

| Name                         | Folder | Type       | Size    | Status    | Error |
|------------------------------|--------|------------|---------|-----------|-------|
| <a href="#">football.jpg</a> | -      | image/jpeg | 56.1 KB | Succeeded | -     |

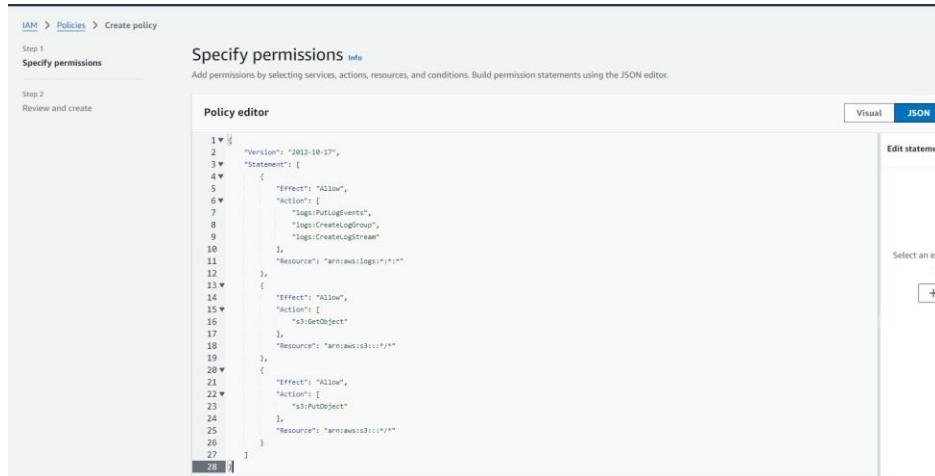
## Step 2: Permissions Setup

- **Create a Permissions Policy:**

Using AWS Identity and Access Management (IAM), I created a custom policy named `LambdaS3Policy` to grant Lambda permissions to read from the source bucket and write to the destination bucket, along with logging to CloudWatch.

### To create the policy (console)

1. Open the [Policies](#) page of the AWS Identity and Access Management (IAM) console.
2. Choose Create policy.
3. Choose the JSON tab, and then paste the following custom policy into the JSON editor.



## Review and create [Info](#)

Review the permissions, specify details, and tags.

### Policy details

#### Policy name

Enter a meaningful name to identify this policy.

Maximum 128 characters. Use alphanumeric and '+,=, @, \_' characters.

#### Description - *optional*

Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and '+,=, @, \_' characters.

Code:

```
{

  "Version": "2012-10-17",

  "Statement": [

    {

      "Effect": "Allow",

      "Action": [

        "logs:PutLogEvents",

        "logs:CreateLogGroup",
```

```

      "logs:CreateLogStream"

    ],

    "Resource": "arn:aws:logs:*:*:*"

  },

  {

    "Effect": "Allow",

    "Action": [

      "s3:GetObject"

    ],

    "Resource": "arn:aws:s3:::*/*"

  },

  {

    "Effect": "Allow",

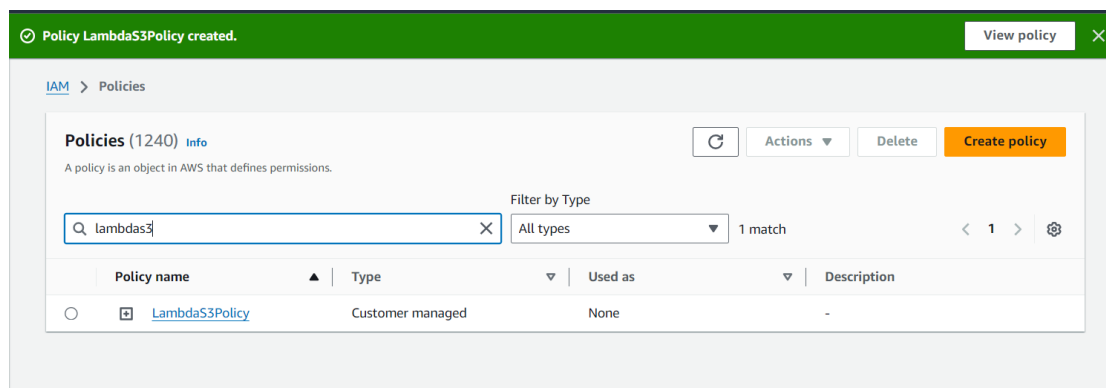
    "Action": [

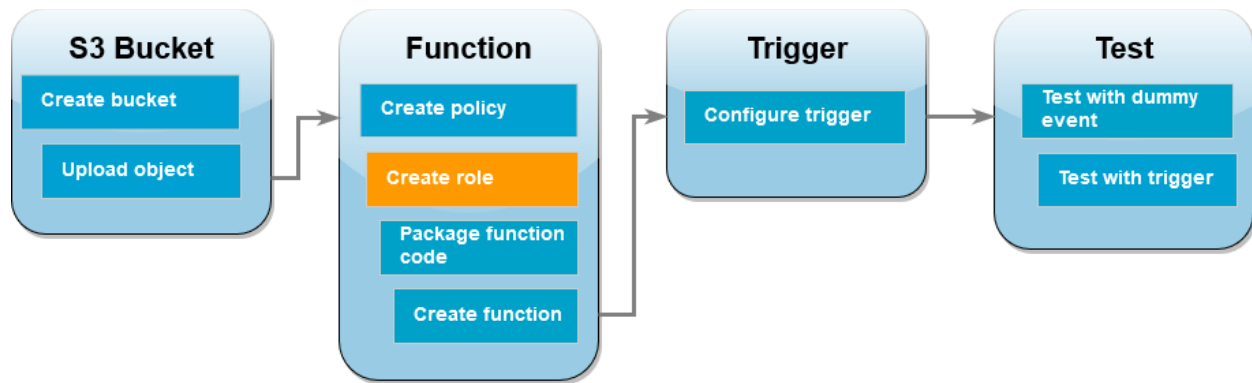
      "s3:PutObject"

    ],

    "Resource": "arn:aws:s3:::*/*" } }

```



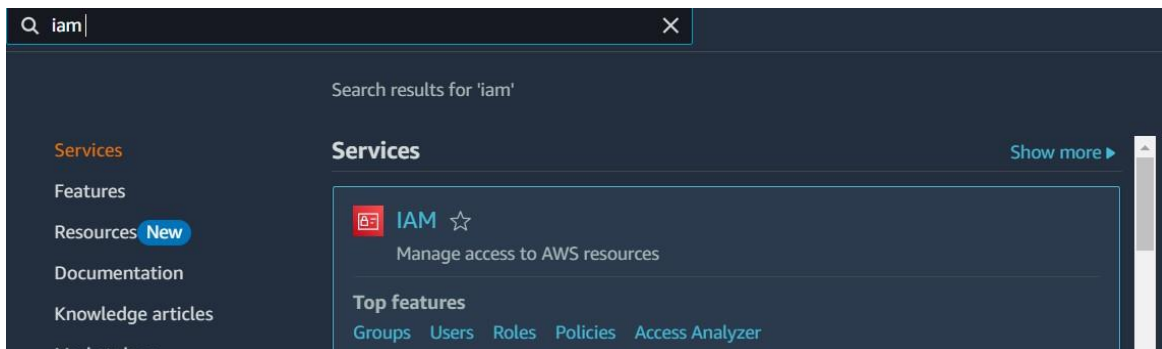


## Creating an Execution Role and Attaching Permissions Policy

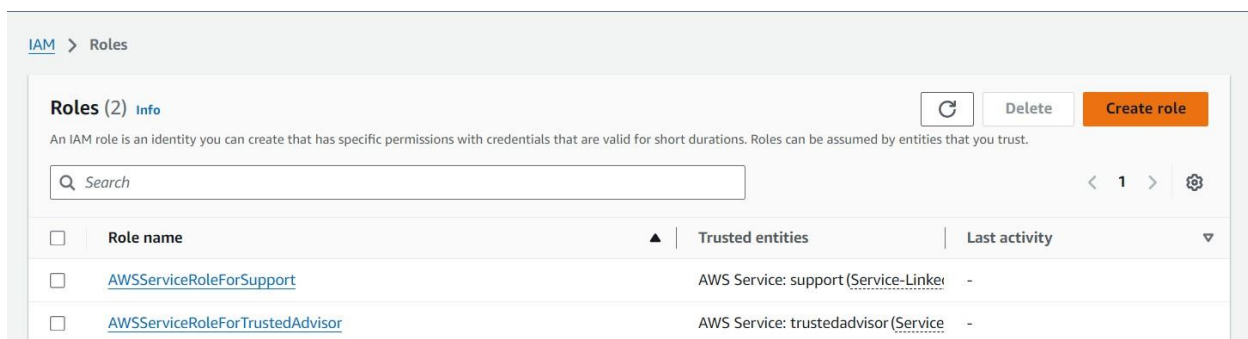
To enable the Lambda function to interact with the Amazon S3 buckets, it requires an **IAM execution role**. This role grants the necessary permissions for the function to read from the source bucket and write to the destination bucket, which are essential for the image resizing process.

### Steps to Create an Execution Role and Attach a Permissions Policy (Using the AWS Console):

1. Open the **IAM Roles** page in the AWS Management Console.



2. Click on **Create role**.



- Under **Trusted entity type**, select **AWS service**.

The screenshot shows the 'Trusted entity type' section with five radio button options: 'AWS service' (selected), 'AWS account', 'Web identity', 'SAML 2.0 federation', and 'Custom trust policy'. Below this is the 'Use case' section, which includes a dropdown menu for 'Service or use case' set to 'Lambda' and a 'Use case' section with 'Lambda' selected.

**Trusted entity type**

- ☒ **AWS service**  
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**  
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**  
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**  
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**  
Create a custom trust policy to enable others to perform actions in this account.

**Use case**  
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case  
Lambda

Choose a use case for the specified service.  
Use case  
☒ **Lambda**  
Allows Lambda functions to call AWS services on your behalf.

- For the **Use case**, choose **Lambda**.
- Click on **Next**.
- In the **Policy search box**, type **LambdaS3Policy** (the name of the policy you created in the earlier step).
- From the search results, select the check box next to **LambdaS3Policy** to attach it to the role.

The screenshot shows the 'Add permissions' section with a search bar containing 'lambdas3'. The search results table shows 'LambdaS3Policy' selected. There are navigation buttons at the bottom: 'Cancel', 'Previous', and 'Next'.

**Add permissions** [Info](#)

**Permissions policies (1/956)** [Info](#)  
Choose one or more policies to attach to your new role.

Filter by Type  
Search: lambdas3 X All types 1 match

| <input checked="" type="checkbox"/> | Policy name                    | Type             | Description |
|-------------------------------------|--------------------------------|------------------|-------------|
| <input checked="" type="checkbox"/> | <a href="#">LambdaS3Policy</a> | Customer managed |             |

► **Set permissions boundary - optional**

Cancel Previous **Next**

- Click **Next** to proceed.

9. Under **Role details**, enter **LambdaS3Role** as the **Role name**.

### Name, review, and create

Role details

**Role name**  
Enter a meaningful name to identify this role.

LambdaS3Role

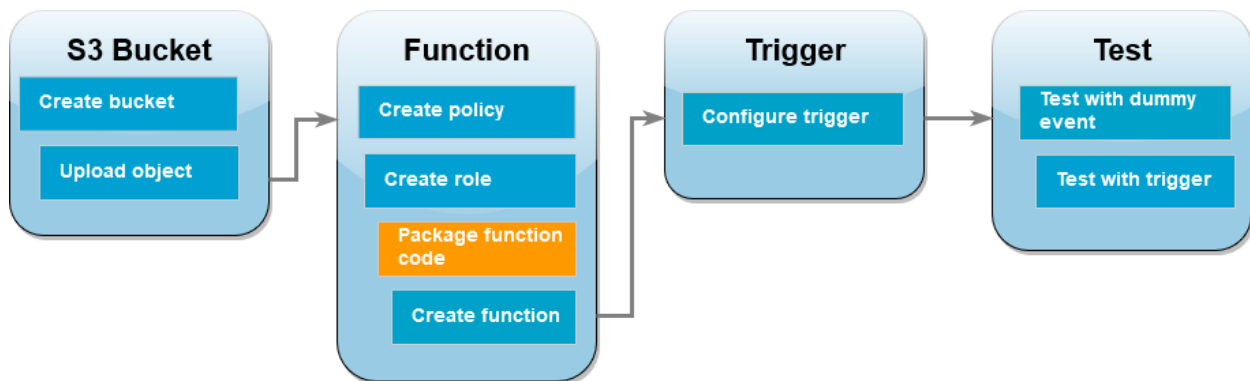
Maximum 64 characters. Use alphanumeric and '+', '@', '-' characters.

**Description**  
Add a short explanation for this role.

Allows Lambda functions to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: '\_', '+', '.', '@', '-', '/', '[', ']', '#', '\$', '%', '^', '&', '\*', '!', ':', ';"<>'.

10. Finally, click **Create role**.



### Steps to Create the Deployment Package (Python)

#### Save the Lambda Function Code:

- Save the provided Python code as `lambda_function.py`.

Code:

```
import boto3
```

```
import os
```

```
import sys

import uuid

from urllib.parse import unquote_plus

from PIL import Image

import PIL.Image

s3_client = boto3.client('s3')

def resize_image(image_path, resized_path):
    with Image.open(image_path) as image:
        image.thumbnail(tuple(x / 2 for x in image.size))
        image.save(resized_path)

def lambda_handler(event, context):
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = unquote_plus(record['s3']['object']['key'])
        tmpkey = key.replace('/', '')
        download_path = '/tmp/{}'.format(uuid.uuid4(), tmpkey)
        upload_path = '/tmp/resized-{}'.format(tmpkey)
        s3_client.download_file(bucket, key, download_path)
        resize_image(download_path, upload_path)
        s3_client.upload_file(upload_path, '{}-resized'.format(bucket), 'resized-
{}'.format(key))
```

```
lambda_function.py 4 X
lambda_function.py > lambda_handler
1  import boto3
2  import os
3  import sys
4  import uuid
5  from urllib.parse import unquote_plus
6  from PIL import Image
7  import PIL.Image
8
9  s3_client = boto3.client('s3')
10
11 def resize_image(image_path, resized_path):
12     with Image.open(image_path) as image:
13         image.thumbnail(tuple(x / 2 for x in image.size))
14         image.save(resized_path)
15
16 def lambda_handler(event, context):
17     for record in event['Records']:
18         bucket = record['s3']['bucket']['name']
19         key = unquote_plus(record['s3']['object']['key'])
20         tmpkey = key.replace('/', '')
21         download_path = '/tmp/{}'.format(uuid.uuid4(), tmpkey)
22         upload_path = '/tmp/resized-{}'.format(tmpkey)
23         s3_client.download_file(bucket, key, download_path)
24         resize_image(download_path, upload_path)
25         s3_client.upload_file(upload_path, '{}-resized'.format(bucket), 'resized-{}'.format(key))
```

## Install Dependencies:

- In the directory with `lambda_function.py`, create a folder named `package`  
`mkdir package`

Install Pillow and Boto3 libraries inside the `package` folder using the command:

```
pip install --platform manylinux2014_x86_64 --target=package --implementation cp --python-version 3.12 --only-binary=:all: --upgrade pillow boto3
```

```
PS D:\D15C36AtharvNikam\documentation> mkdir package1

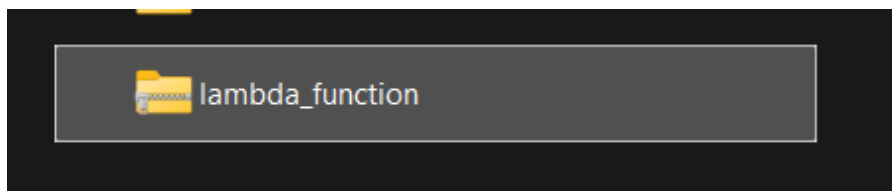
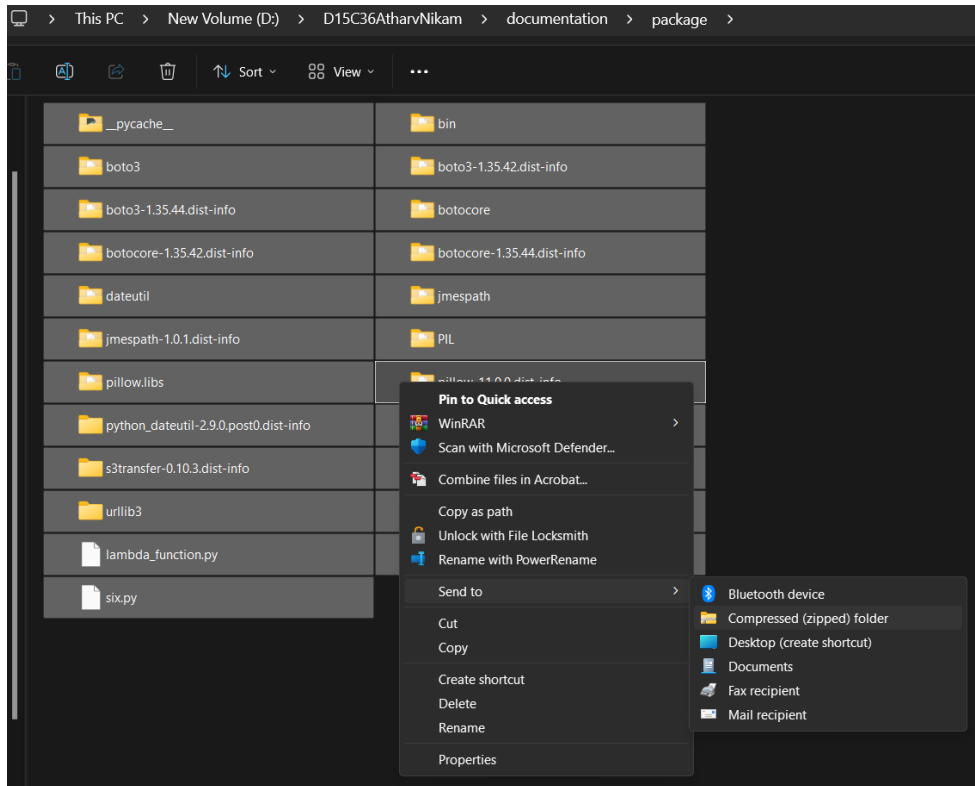
Directory: D:\D15C36AtharvNikam\documentation

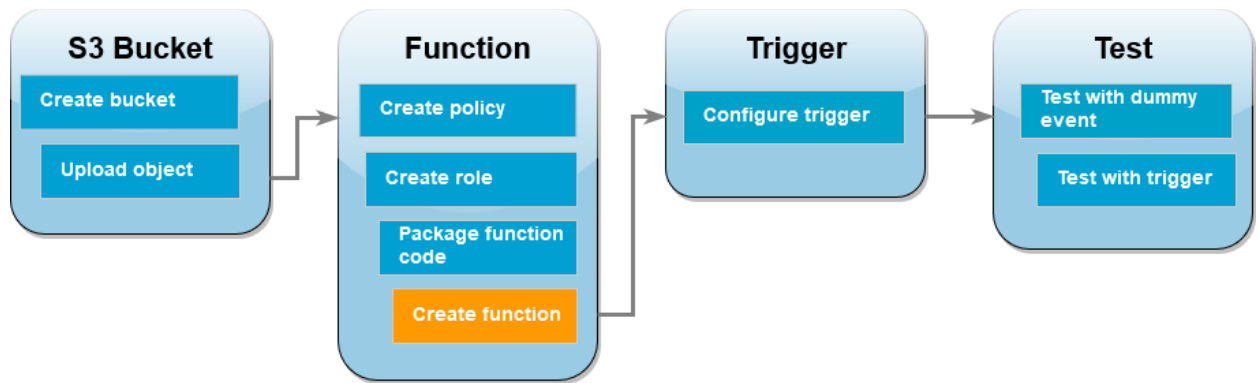
Mode                LastWriteTime         Length Name
----                -
d-----          19-10-2024  10:00 PM                package1

PS D:\D15C36AtharvNikam\documentation> pip install `
>> --platform manylinux2014_x86_64 `
>> --target=package `
>> --implementation cp `
>> --python-version 3.12 `
>> --only-binary=:all: `
>> --upgrade `
>> pillow boto3
Collecting pillow
  Using cached pillow-11.0.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (9.1 kB)
Collecting boto3
  Using cached boto3-1.36.1-py3-none-any.whl.metadata (6.9 kB)
```

## Package the Application:

Use any ZIP tool to create `lambda_function.zip` with `lambda_function.py` and the `package` folder contents.

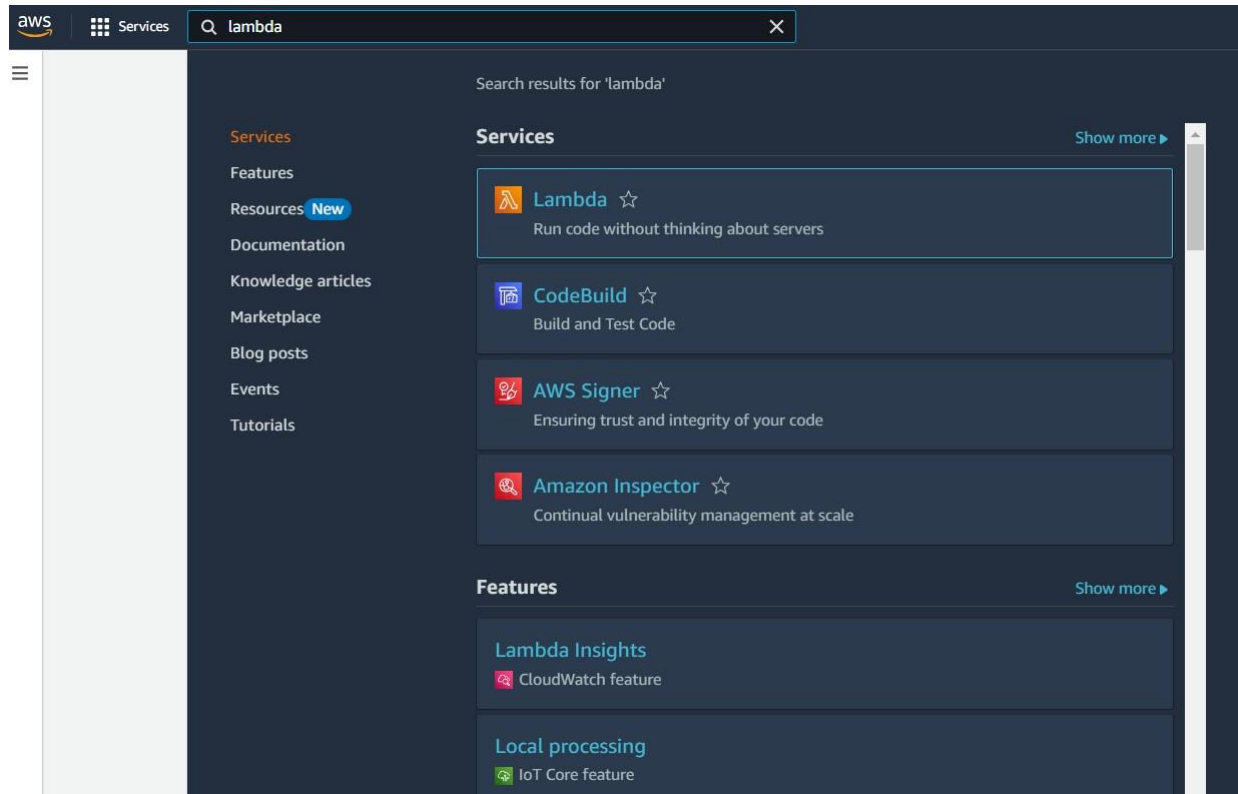




## Steps to Create the Lambda Function (Console)

### Access the Lambda Console:

- Open the **Functions** page of the AWS Lambda console.
- Ensure you are in the same AWS Region where you created your S3 bucket (you can change the region from the drop-down menu).



## Create a New Function:

Choose **Create function**.

Select **Author from scratch**.

Fill in the **Basic information**:

- **Function name:** `resizeImage`
- **Runtime:** Select either **Python 3.12**
- **Architecture:** Choose **x86\_64**.

[Lambda](#) > [Functions](#) > Create function

## Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**  
Start with a simple Hello World example.

☐ **Use a blueprint**  
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**  
Select a container image to deploy for your function.

☐ **Browse serverless app repository**  
Deploy a sample Lambda application from the AWS Serverless Application Repository.

### Basic information

**Function name**  
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (\_).

**Runtime** [Info](#)  
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.



**Architecture** [Info](#)  
Choose the instruction set architecture you want for your function code.

☒ **x86\_64**

☐ **arm64**

In the **Change default execution role** section:

- Expand the tab and select **Use an existing role**.
- Choose the `LambdaS3Role` you created earlier.

Click **Create function**.

**▼ Change default execution role**

**Execution role**  
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions

☒ Use an existing role

☐ Create a new role from AWS policy templates

**Existing role**  
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

**lambda-resizer**

Throttle Copy ARN Actions

✓ The trigger amzn-s3-bbp-source-bucket was successfully added to function lambda-resizer. The function is now receiving events from the trigger.

**▼ Function overview** Info

Export to Application Composer Download

Diagram Template

Diagram showing S3 trigger connected to lambda-resizer function.

Layers (0)

+ Add destination

+ Add trigger

Description

Last modified 15 minutes ago

Function ARN arn:aws:lambda:us-east-1:061051254449:function:lambda-resizer

Function URL Info

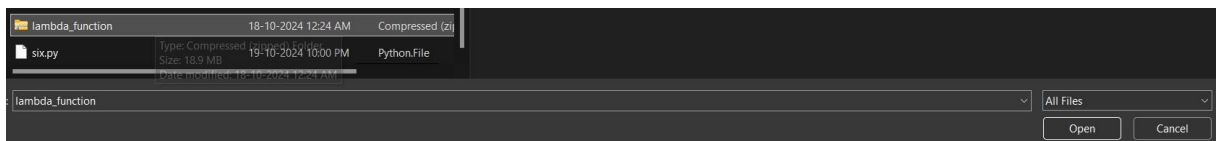
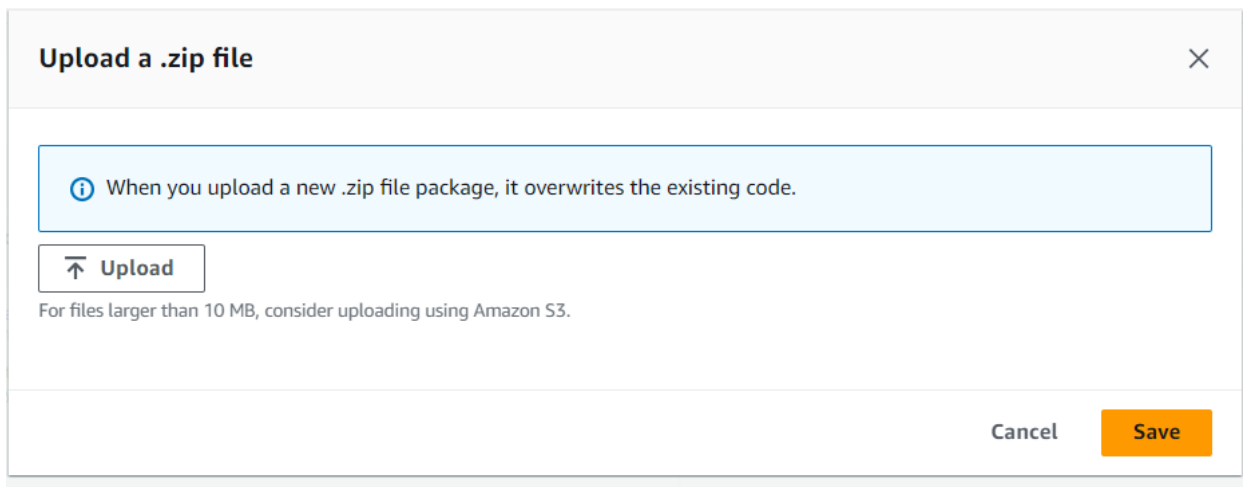
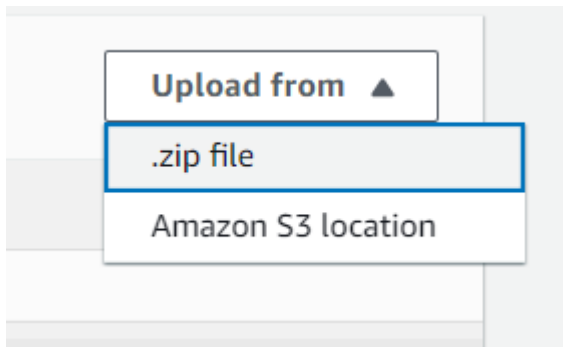
Code Test Monitor Configuration Aliases Versions

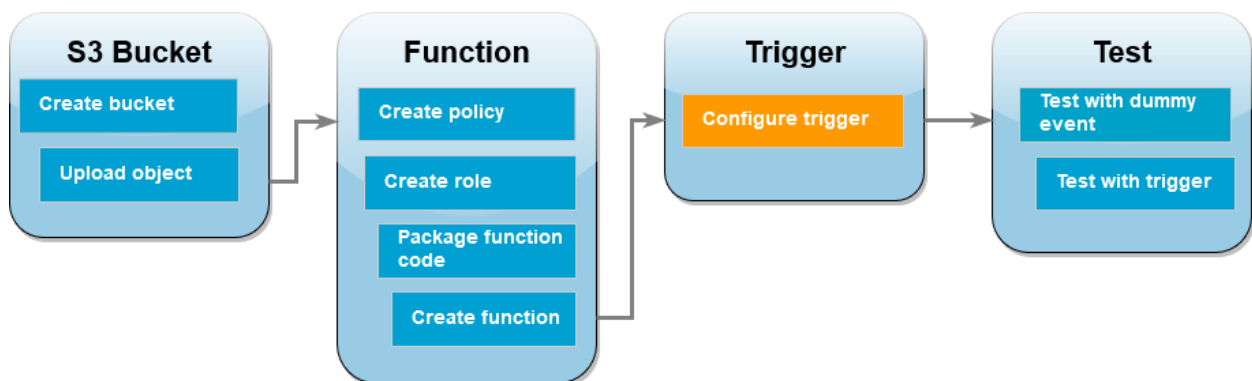
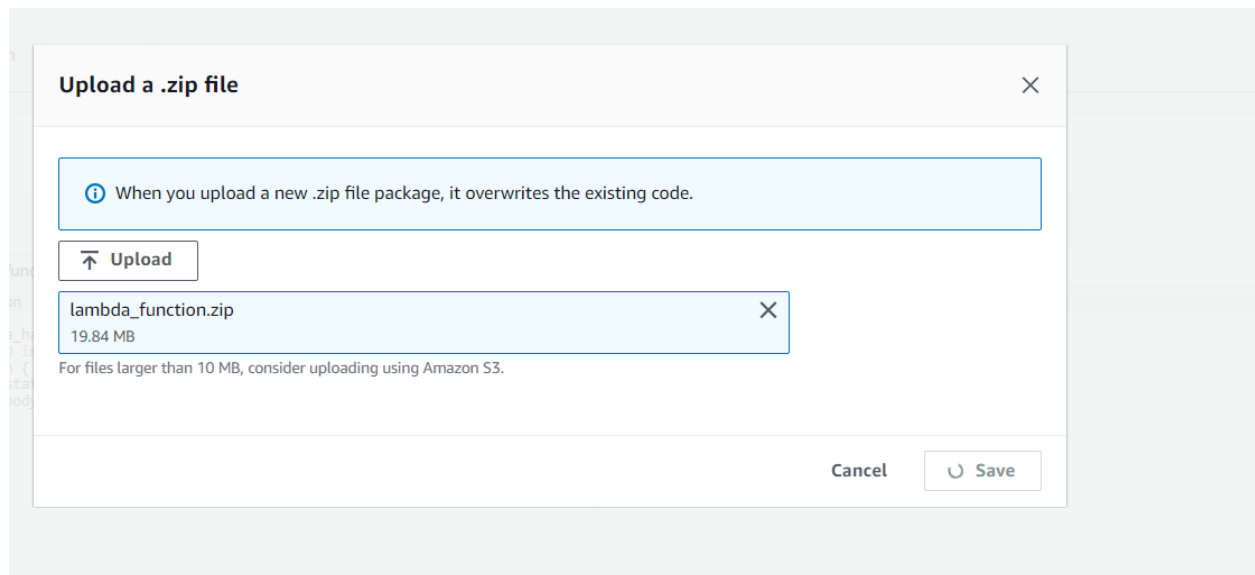
CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

## Upload Function Code:

- In the **Code source** pane, select **Upload from**.
- Choose **.zip file**.
- Select your **.zip** file and click **Open**.
- Click **Save**.





## Steps to Configure the Amazon S3 Trigger (Console)

### Access the Lambda Function:

- Open the **Functions** page in the AWS Lambda console.
- Select your function:resizeImage

### Add S3 Trigger:

- Click on **Add trigger**.
- Choose **S3** as the trigger type.
- Under **Bucket**, select your source bucket.
- Under **Event types**, select **All object create events**.
- Check the box to acknowledge the warning about recursive invocation.

Search

- ☒ All object create events
  - ☒ All object create events
  - ☐ PUT
  - ☐ POST
  - ☐ COPY
  - ☐ Multipart upload completed
- ☐ All object delete events
  - ☐ All object delete events
  - ☐ Permanently deleted
  - ☐ Delete marker created
- ☐ Restore from Glacier initiated
  - ☐ Restore from Glacier initiated
  - ☐ Restore from Glacier completed

Prefix - optional

must be in the same region as the

ily set up a prefix or suffix for an prefixes or suffixes that could mat

All object create events X

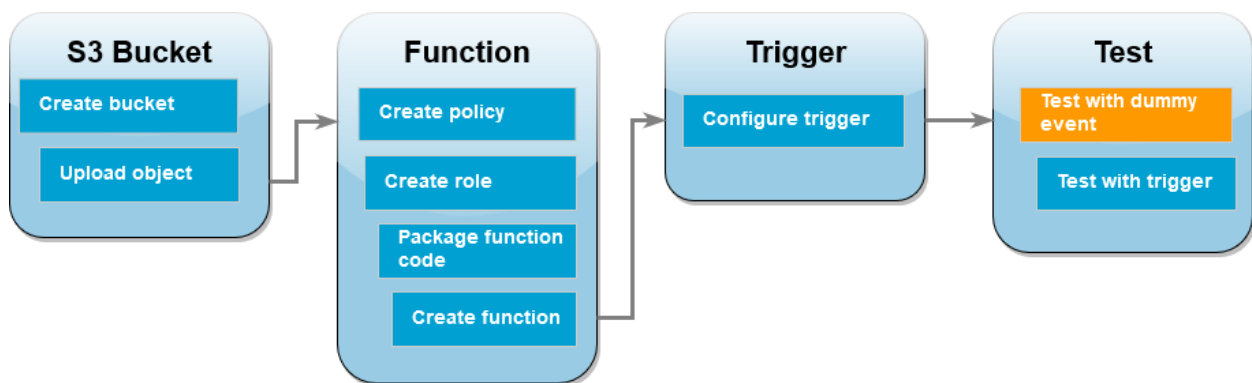
more

☒ I acknowledge that using the same S3 bucket for both input and output is not recommended and that this configuration can cause recursive invocations, increased Lambda usage, and increased costs.

Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Cancel

Add



## Steps to Test Your Lambda Function with a Dummy Event (Console)

### 1. Access Your Lambda Function:

- Open the **Functions** page in the AWS Lambda console.

- 0 Select your function: **CreateThumbnail**.

The screenshot shows the 'Test event' configuration page in the AWS Lambda console. At the top, there are tabs for 'Code', 'Test' (which is selected), 'Monitor', 'Configuration', 'Aliases', and 'Versions'. Below the tabs, the page title is 'Test event' with an 'Info' link. A paragraph explains: 'To invoke your function without saving an event, configure the JSON event, then choose Test.' Under 'Test event action', there is a radio button selected for 'Create new event'. Below that, the 'Event name' field contains 'myTestEvent', with a note: 'Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.' Under 'Event sharing settings', there are two radio buttons: 'Private' (selected) and 'Shareable'. The 'Private' option has a note: 'This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)'. The 'Shareable' option has a note: 'This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)'.

### Create a Test Event:

- Go to the **Test** tab.
- In the **Test event** pane, select **Create new event**.
- **Event name:** Enter `myTestEvent`.
- **Template:** Select **S3 Put**.

The screenshot shows a dropdown menu for selecting a template. The text 's3-put' is entered into the search field. Below the search field, there is a list of options: 'S3 Batch Operat', 'S3 Delete', and 'S3 Put'. The 'S3 Put' option is highlighted with a blue background.

Customize Event Parameters:

```
{
  "Records": [
    {
      "eventVersion": "2.0",
      "eventSource": "aws:s3",
      "awsRegion": "eu-north-1",
      "eventTime": "1970-01-01T00:00:00.000Z",
      "eventName": "ObjectCreated:Put",
      "userIdentity": {
        "principalId": "EXAMPLE"
      },
      "requestParameters": {
        "sourceIPAddress": "127.0.0.1"
      },
      "responseElements": {
        "x-amz-request-id": "EXAMPLE123456789",
        "x-amz-id-2":
"EXAMPLE123/5678abcdefghijklmbdaisawesome/mnopqrstuvwxyzABCDEFGH"
      },
      "s3": {
        "s3SchemaVersion": "1.0",
        "configurationId": "testConfigRule",
        "bucket": {
          "name": "amzn-s3-sourcr-bucket",
          "ownerIdentity": {
            "principalId": "EXAMPLE"
          },
          "arn": "arn:aws:s3:::amzn-s3-demo-bucket"
        },
        "object": {
          "key": "tiger",
          "size": 1024,
          "eTag": "0123456789abcdef0123456789abcdef",
          "sequencer": "0A1B2C3D4E5F678901"
        }
      }
    }
  ]
}
```

**Save and Test:**

- Click **Save**.

- In the **Test event** pane, choose **Test**.

### Verify Resized Image:

- Open the **Buckets** page in the Amazon S3 console.
- Select your target bucket and check the **Objects** pane for the resized image.

Code

Test

Monitor

Configuration

Aliases

Versions

✖

Executing function: failed ([logs](#))  
▶ Details

Code

Test

Monitor

Configuration

Aliases

Versions

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

General configuration

Info

Edit

Description

-

Memory

128 MB

Ephemeral storage

512 MB

Timeout

0 min 3 sec

SnapStart

None

Info

Timeout

0

min

15

sec

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Use an existing role

☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

service-role/resizeImage-role-zlnpuppu

View the [resizeImage-role-zlnpuppu](#) role on the IAM console.

Cancel

Save

✔

Successfully updated the function **resizeImage**.

Code
Test
Monitor
Configuration
Aliases
Versions



Executing function: succeeded ([logs](#))

▶ Details

Objects (2)

Info

↻

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

<

1

>

⚙

| <input type="checkbox"/> | Name                                                                                                                 | Type | Last modified                          | Size   | Storage class |
|--------------------------|----------------------------------------------------------------------------------------------------------------------|------|----------------------------------------|--------|---------------|
| <input type="checkbox"/> |  <a href="#">indian_flafpng.png</a> | png  | October 25, 2024, 13:27:34 (UTC+05:30) | 1.6 KB | Standard      |

```

graph LR
    subgraph S3_Bucket [S3 Bucket]
        direction TB
        S3_Bucket_1[Create bucket]
        S3_Bucket_2[Upload object]
    end
    subgraph Function
        direction TB
        Function_1[Create policy]
        Function_2[Create role]
        Function_3[Package function code]
        Function_4[Create function]
    end
    subgraph Trigger
        direction TB
        Trigger_1[Configure trigger]
    end
    subgraph Test
        direction TB
        Test_1[Test with dummy event]
        Test_2[Test with trigger]
    end
    S3_Bucket --> Function
    Function --> Trigger
    Trigger --> Test
  
```

## Steps to Test Your Lambda Function Using the Amazon S3 Trigger (Console)

- Upload an Image to Your Source Bucket:**
  - Open the **Buckets** page in the Amazon S3 console.

- 0 Select your **source bucket**.
- 0 Click on **Upload**.
- 0 Choose **Add files** and select the image file (any **.jpg** or **.png**).
- 0 Click **Open**, then choose **Upload**.
- 0

### Objects (2) [Info](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

<
1
>

| <input type="checkbox"/> | Name                               | Type | Last modified                          | Size    | Storage class |
|--------------------------|------------------------------------|------|----------------------------------------|---------|---------------|
| <input type="checkbox"/> | <a href="#">indian_flafpng.png</a> | png  | October 25, 2024, 13:27:34 (UTC+05:30) | 1.6 KB  | Standard      |
| <input type="checkbox"/> | <a href="#">tiger.jpg</a>          | jpg  | October 25, 2024, 14:51:13 (UTC+05:30) | 58.4 KB | Standard      |



Objects (2) Info

↻

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

| <input type="checkbox"/> | Name                                       | Type | Last modified                          | Size    | Storage class |
|--------------------------|--------------------------------------------|------|----------------------------------------|---------|---------------|
| <input type="checkbox"/> | <a href="#">resized-indian_flafpng.png</a> | png  | October 25, 2024, 13:27:38 (UTC+05:30) | 643.0 B | Standard      |
| <input type="checkbox"/> | <a href="#">resized-tiger.jpg</a>          | jpg  | October 25, 2024, 14:51:17 (UTC+05:30) | 14.4 KB | Standard      |



**Conclusion:** This project shows how AWS Lambda and S3 can automatically resize images without needing to manage any servers. The process is simple and efficient, with images being resized when uploaded and stored in a separate bucket. It improves loading times and works well for websites and apps. Hosting the frontend on Netlify makes the deployment easier, and overall, the system works smoothly for the intended purpose.

